

BOXX Chatbot Test API

A minimal HTTP API for **automated testing of the BOXX chatbot**. Create a session, send it messages, and read the bot's replies back — no WhatsApp, no dashboard, no login flow. Every message runs through the **exact same pipeline a real WhatsApp message does**, so replies reflect true production behaviour. Nothing is ever sent to Meta/WhatsApp; the conversation is fully isolated.

- **Base URL:** `https://boxxv2.shunya.ai`
- **Auth:** a single shared **API key** sent on every request.
- **API key:** `boxx-qa-e1ae28770694f29b7ebc2cab3743438a`
- **Content type:** `application/json`. All responses are wrapped in `{ "status": "success", "data": { ... } }`.

Authentication

Send the API key on **every** request, using any one of these (pick whichever your tooling makes easiest):

```
X-API-Key: boxx-qa-e1ae28770694f29b7ebc2cab3743438a
```

or

```
Authorization: Bearer boxx-qa-e1ae28770694f29b7ebc2cab3743438a
```

or as a query parameter: `?api_key=boxx-qa-e1ae28770694f29b7ebc2cab3743438a`

A missing/invalid key returns **401**.

Quick start (2 calls)

```
BASE=https://boxxv2.shunya.ai
KEY=boxx-qa-e1ae28770694f29b7ebc2cab3743438a

# 1) Create a session -> get a session_id
SID=$(curl -s -H "X-API-Key: $KEY" -X POST "$BASE/api/test/session" \
  -H 'Content-Type: application/json' \
  -d '{"language":"en","profile_name":"QA Bot"}' | jq -r '.data.session_id')
```

```
# 2) Send a message -> get the bot's reply
curl -s -H "X-API-Key: $KEY" -X POST "$BASE/api/test/session/$SID/message" \
  -H 'Content-Type: application/json' \
  -d '{"message": "I got an SMS saying my KYC will expire, click this link"}' | jq
```

Keep calling endpoint 2 with the same `session_id` to continue the conversation — the bot remembers context across turns, exactly as it would on WhatsApp.

Endpoints

GET `/api/test/health`

Sanity check — confirms your API key works and returns the org the sessions belong to.

```
curl -s -H "X-API-Key: $KEY" https://boxxv2.shunyalabs.ai/api/test/health | jq
```

```
{
  "status": "success",
  "data": { "status": "ok", "organization_id": "...", "time": "2026-07-28T10:00:00Z"
}
```

POST `/api/test/session`

Create a new, isolated conversation. **Body is optional** — send `{}` for defaults.

Field	Type	Default	Notes
<code>language</code>	string	<code>"en"</code>	<code>en</code> , <code>hi</code> (Hindi), <code>hing</code> (Hinglish), <code>bn</code> , ...
<code>profile_name</code>	string	<code>"QA Test User"</code>	Display name for the test user.
<code>phone</code>	string	<code>"simulation"</code>	Cosmetic label only; a synthetic phone is always generated internally.
<code>metadata</code>	object	—	Free-form; echoed back on the contact.

```
curl -s -H "X-API-Key: $KEY" -X POST https://boxxv2.shunyalabs.ai/api/test/session
  -H 'Content-Type: application/json' -d '{"language": "hing", "profile_name": "Ravi"
```

```
{
```

```
"status": "success",
"data": {
  "session_id": "7c2e...-...-",
  "profile_name": "Ravi",
  "language": "hing",
  "whatsapp_account": "simulation-account",
  "created_at": "2026-07-28T10:00:00Z"
}
```

Save `data.session_id` — you need it for every message.

POST /api/test/session/{id}/message

Send one user message into the session and get the bot's reply/replies **for that turn**.

Field	Type	Notes
<code>message</code>	string	The user's text. Required unless <code>button_id</code> is set.
<code>button_id</code>	string	Simulate tapping a WhatsApp reply button (see Buttons).
<code>button_title</code>	string	Optional label for the tapped button.

```
curl -s -H "X-API-Key: $KEY" -X POST https://boxxv2.shunyalabs.ai/api/test/session
-H 'Content-Type: application/json' \
-d '{"message":"someone from the bank called and asked for my OTP"}' | jq
```

```
{
  "status": "success",
  "data": {
    "session_id": "7c2e...",
    "user_message": "someone from the bank called and asked for my OTP",
    "reply": "That sounds like a scam – a real bank will never ask for your OTP...",
    "replies": [
      "That sounds like a scam – a real bank will never ask for your OTP...",
      "Here's what to do right now: 1) Don't share any code..."
    ],
    "reply_count": 2,
    "latency_ms": 2831,
    "messages": [ { "id": "...", "type": "text", "content": "...", "created_at": "..." } ],
    "analysis": {
      "journey": "check_scam",
      "classification": "vishing",
      "emotion": "anxious",
      "emotion_language": "en"
    }
  }
}
```

```
}
```

Key fields for automation:

- **reply** — all reply bubbles joined with blank lines (the simplest thing to assert on).
- **replies** — array of individual bubbles, in order (the bot may send several).
- **analysis** — what the pipeline detected this turn: **journey** (route), **classification** (scam type), **emotion**, **loss_assessment**. Handy for asserting *why*, not just *what*. Any field may be empty on a given turn.

The bot can reply with **more than one bubble** per message. **replies** gives you each one; **reply** gives you them joined. **reply_count** is how many had text.

GET /api/test/session/{id}

Fetch the full transcript so far (both sides), oldest first.

```
curl -s -H "X-API-Key: $KEY" https://boxxv2.shunyalabs.ai/api/test/session/$SID |
```

```
[
  { "role": "user", "content": "someone asked for my OTP", "type": "text", "create
    { "role": "bot", "content": "That sounds like a scam...", "type": "text", "creat
  ]
```

POST /api/test/quick

One-shot convenience: creates a throwaway session, sends one message, returns the reply **and** the new **session_id** (so you can continue if you want). Good for stateless single-prompt checks.

```
curl -s -H "X-API-Key: $KEY" -X POST https://boxxv2.shunyalabs.ai/api/test/quick \
  -H 'Content-Type: application/json' -d '{"message":"is 1930 a real helpline?"}'
```

Buttons

The very first turn of a fresh session usually returns a short **disclaimer** and asks the user to tap **"I Agree"** before the bot proceeds. To move past it in automation, send the button instead of text:


```
curl -s -H "X-API-Key: $KEY" -X POST https://boxxv2.shunyalabs.ai/api/test/session \
-H 'Content-Type: application/json' \
-d '{"button_id":"agree_disclaimer","button_title":"I Agree"}' | jq '.data.reply'
```

The exact `button_id` for any interactive prompt is in that reply's `messages[].interactive_data`. If your test flow doesn't care about the disclaimer, you can also just send your real message — the bot re-prompts and continues on the next turn.

End-to-end example (Python)

```
import requests

BASE = "https://boxxv2.shunyalabs.ai"
HEADERS = {"X-API-Key": "boxx-qa-e1ae28770694f29b7ebc2cab3743438a"}

def create_session(language="en"):
    r = requests.post(f"{BASE}/api/test/session", headers=HEADERS, json={"language": language})
    r.raise_for_status()
    return r.json()["data"]["session_id"]

def send(session_id, message):
    r = requests.post(f"{BASE}/api/test/session/{session_id}/message",
                      headers=HEADERS, json={"message": message}, timeout=30)
    r.raise_for_status()
    return r.json()["data"]

sid = create_session(language="en")

turn = send(sid, "I got a call saying my electricity will be cut tonight, pay now")
print("BOT:", turn["reply"])
print("detected scam type:", turn["analysis"].get("classification"))

# assert the bot flagged it as a scam and didn't leak an unexpected phone number
assert "scam" in turn["reply"].lower() or turn["analysis"].get("classification")
```

Notes & behaviour

- **Isolation** — every session is a synthetic contact tagged `simulation`. It never touches real WhatsApp users and never sends to Meta. You can create as many as you like.
- **Latency** — a turn is synchronous: the response returns only after the bot has finished. Simple turns are fast (tens of ms); turns that hit the LLM/graph typically take **2–9 s**. Set your HTTP client timeout to ≥ 30 s.

- **State** — context (language, emotion, scam classification, journey) persists across turns within a `session_id`, just like a real chat. Use a fresh session per independent test case.
- **Replies are the real thing** — the same guardrails, grounding, and language handling that run in production run here. If the bot would stay silent in prod (rare), `replies` can be empty.
- **Errors** — non-2xx responses use `{ "status": "error", "message": "..." }`. Common: `401` (bad/missing API key), `404` (unknown `session_id`), `400` (empty message).

Configuration

For the platform/ops team — all optional, set as environment variables on the `boxx_app` service:

Variable	Default	Purpose
<code>TEST_API_ENABLED</code>	<code>true</code>	Set to <code>false</code> to disable all <code>/api/test/*</code> endpoints (returns 404).
<code>TEST_API_KEY</code>	<code>boxx-qa-e1ae28770694f29b7ebc2cab3743438a</code>	The shared API key. Change this to rotate the key.
<code>TEST_API_ORG_ID</code>	oldest org in the DB	UUID of the organization the test sessions belong to.
<code>TEST_API_USERNAME</code> / <code>TEST_API_PASSWORD</code>	<code>qa</code> / <code>boxx-test</code>	Optional HTTP Basic Auth fallback (API key is the primary method).

The endpoints are JWT-exempt by design (API key only) so QA tooling stays simple. They only ever create isolated simulation data, but if you don't want them reachable in a given environment, set `TEST_API_ENABLED=false`. To rotate the key, set `TEST_API_KEY` and restart `boxx_app`.